

Foundations of 4D/6D Object Capture for Virtual Environments

Motivation and Linear Transformations



February 23, 2026

Goal: We want to digitize the real world

Question: Why?



Advertisement
→ Fashion

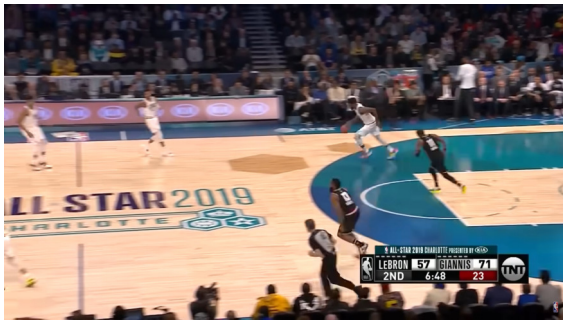


(AR/VR) Training
→ Medicine



TV Broadcasting
→ Sports

Currently: 2D Videos



https://youtu.be/OrH7JHHn_q4?t=233

The Good:

- ▶ Universally accessible
- ▶ Cheap to produce

The Bad:

- ▶ Viewpoint tied to camera
- ▶ "Flat" appearance

→ Limited immersion

Next Step: Volumetric Videos

The Good:

- ▶ Free choice of viewpoint
- ▶ True 3D perception (AR / VR)

The Mixed:

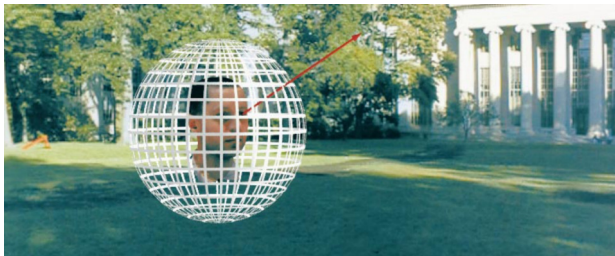
- ▶ Limited accessibility
- ▶ Still high costs

→ Offers new creative possibilities



https://youtu.be/L_VGM_s777I

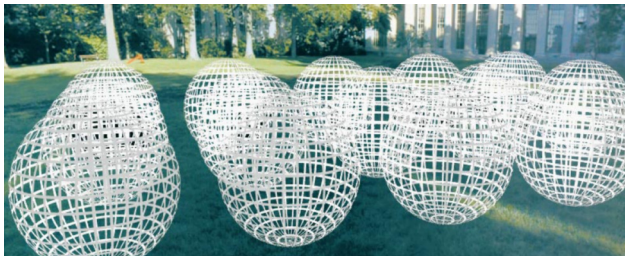
Question: How can we **mathematically characterize** the difference between 2D Videos and Volumetric Videos?



A **2D Video** $P(\theta, \phi, \lambda, t)$ is the light observed ...

- ▶ ... from every view direction (θ, ϕ)
- ▶ ... from every wavelength λ
- ▶ ... at every time t

Question: How can we **mathematically characterize** the difference between 2D Videos and Volumetric Videos?



A **Volumetric Video** $P(\theta, \phi, \lambda, t, V_x, V_y, V_z)$ is the light observed . . .

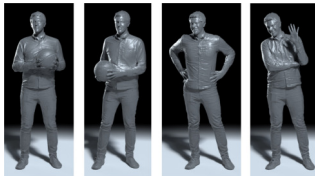
- ▶ . . . from every view direction (θ, ϕ)
- ▶ . . . from every wavelength λ
- ▶ . . . at every time t
- ▶ . . . at every position (V_x, V_y, V_z) in space

Observation: The **Plenoptic Function** $P(\theta, \phi, \lambda, t, V_x, V_y, V_z)$ describes everything we can "see"



3D Reconstruction

- ▶ Geometry of objects
- ▶ (RGB texture)



4D Reconstruction

- ▶ Geometry of objects ...
- ▶ ... over time
- ▶ (RGB texture)

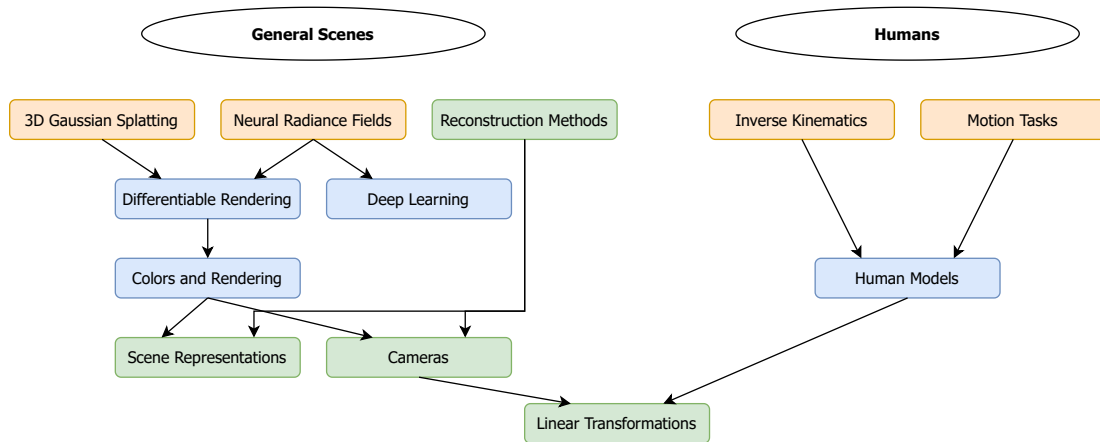


6D Reconstruction

- ▶ Geometry of objects ...
- ▶ ... over time
- ▶ View-dependent color
→ appearance

→ The GOAL of the course **Foundations of 4D/6D Object Capture for Virtual Environments!**

Task: Learn about 3D/4D/6D Reconstruction in **Week 1**, **Week 2**, **Week 3**



Linear Transformations



Definition (Linear Transformation)

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is called *linear* if for all vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and scalars $\lambda \in \mathbb{R}$:

$$f(\lambda \mathbf{x} + \mathbf{y}) = \lambda f(\mathbf{x}) + f(\mathbf{y})$$

We can write linear transformations as $n \times n$ matrices.

Example: In $\mathbb{R}^2$:

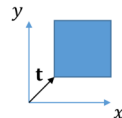
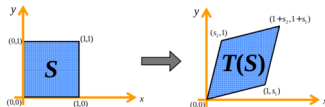
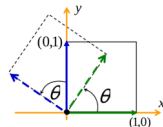
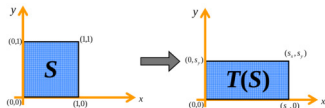
$$f(\mathbf{x}) = \mathbf{T} \cdot \mathbf{x} = \begin{pmatrix} T_{11} & T_{12} \\ T_{21} & T_{22} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} T_{11} \cdot x_1 + T_{12} \cdot x_2 \\ T_{21} \cdot x_1 + T_{22} \cdot x_2 \end{pmatrix}$$

Note: A linear transformation can also be applied simultaneously on multiple vectors by writing the vectors as a matrix and using matrix-matrix multiplication:

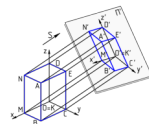
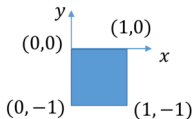
$$\left[\mathbf{T} \cdot \mathbf{x} \mid \mathbf{T} \cdot \mathbf{y} \mid \cdots \right] = \mathbf{T} \cdot \left[\mathbf{x} \mid \mathbf{y} \mid \cdots \right]$$

Types of linear transformations:

- Scaling
- Shearing
- Reflection/Mirroring
- Rotation
- Translation (not directly)
- Projection



→ We will be working a lot with rotations and translations

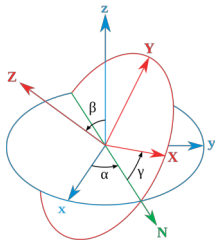


In 2D, a rotation can be described with a single rotation angle:

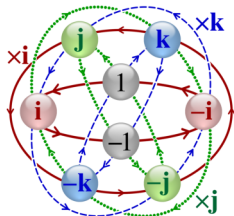
$$R_{2D} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}$$

In 3D, a rotation has **3 degrees of freedom** and can be described in different representations:

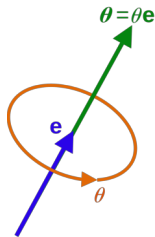
Euler Angles



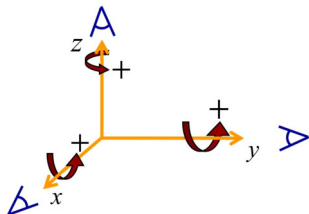
Quaternions



Axis-Angle with Rodrigues' formula



Rotations around one of the coordinate axes can be stated as:



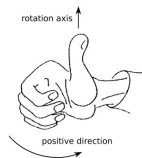
$$x \text{ axis: } \mathbf{R}_x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{pmatrix}$$

$$y \text{ axis: } \mathbf{R}_y = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{pmatrix}$$

$$z \text{ axis: } \mathbf{R}_z = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Note: A rotation is called positive in the mathematical sense when:

- ▶ Looking at the rotation axis from $+\infty$ in the direction of the origin ...
- ▶ ...and then rotating *counter-clockwise*.

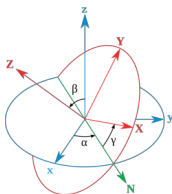


In order to rotate around an arbitrary axis, Euler angles can be used. The transformation is a composition of 3 rotations around the coordinate axes:

$$\mathbf{R}_{\text{euler}}(\alpha, \beta, \gamma) = \mathbf{R}_z(\gamma) \cdot \mathbf{R}_x(\beta) \cdot \mathbf{R}_z(\alpha)$$

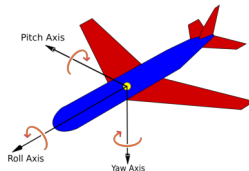
Many conventions with different combinations and sequences between x , y , z axis possible

Proper Euler Angles: z - x - z , x - y - x , ...



→ 1st and 3rd axis repeated

Tait-Bryan Angles: x - y - z , y - z - x , ...



→ 3 distinct axes ("yaw-pitch-roll" in aeronautics)

Note: Rotations are *not commutative*! Order of them is crucial

Problem: Gimbal lock $\rightarrow$ Loose 1 degree of freedom

- ▶ 2nd rotation (β) can make 3rd axis point into direction of 1st axis
- ▶ When 1st and 3rd axis are aligned, they act like a single rotation with $\alpha \pm \gamma$

Proper Euler Angles: z - x - z , x - y - x , ...

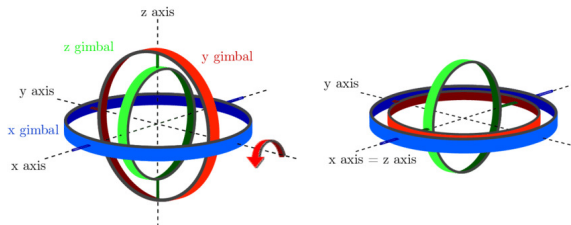
$$\sin(\beta) = 0 \quad \Rightarrow \quad \beta = 0^\circ \vee \beta = 180^\circ$$

Tait–Bryan Angles: x - y - z , y - z - x , ...

$$\cos(\beta) = 0 \quad \Rightarrow \quad \beta = \pm 90^\circ$$

Example: z - y - x with $\beta = 90^\circ$

Solution: Use different representation like quaternions or axis-angle



Given: Two complex numbers $c_1, c_2 \in \mathbb{C}$ with

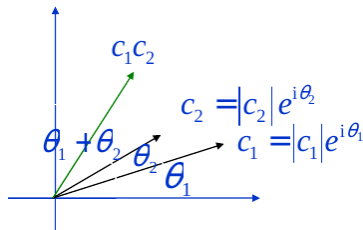
$$c_1 = a_1 + i b_1 \quad c_2 = a_2 + i b_2 \quad i^2 = -1$$

They can be written in polar coordinates as

$$c_1 = |c_1| e^{i\theta_1} \quad c_2 = |c_2| e^{i\theta_2}$$

Thus, the product of them is

$$c_1 c_2 = c_2 c_1 = |c_1| |c_2| e^{i(\theta_1 + \theta_2)}$$



Observation: If $|c_1| = 1$, then $c_1 = e^{i\theta_1} = \cos(\theta_1) + i \sin(\theta_1)$ acts as a rotation in the complex plane

→ Can this be generalized to rotations in 3 orthogonal planes?

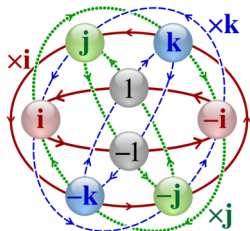
Definition (Quaternion space)

The space of quaternions $\mathbb{H}$ is defined as the set

$$\mathbb{H} = \{w + ix + jy + kz \mid w, x, y, z \in \mathbb{R}\}$$

with imaginary numbers i, j, k and $i^2 = j^2 = k^2 = ijk = -1$.

$\cdot$	1	i	j	k
1	1	i	j	k
i	i	-1	k	-j
j	j	-k	-1	i
k	k	j	-i	-1



Note: A quaternion with $w = 0$ is also called *pure imaginary* quaternion.

Quaternions are the 4D analogue to complex numbers

→ So a unit quaternion $q \in \mathbb{H}$ with $|q| = 1$ represents a rotation under multiplication

Question: How can we write a *specific* rotation with quaternions?

Theorem (Quaternion rotation)

Given a unit vector $\mathbf{r}_0 = (r_x, r_y, r_z)^T \in \mathbb{R}^3$ with $\|\mathbf{r}_0\| = 1$ and an angle $\theta \in \mathbb{R}$, let

- ▶ $q_{\mathbf{r}_0} = e^{(\mathbf{i} r_x + \mathbf{j} r_y + \mathbf{k} r_z) \frac{\theta}{2}} = \cos\left(\frac{\theta}{2}\right) + (\mathbf{i} r_x + \mathbf{j} r_y + \mathbf{k} r_z) \sin\left(\frac{\theta}{2}\right)$ be the respective unit quaternion to $\mathbf{r}_0$,
- ▶ $q_{\mathbf{r}_0}^{-1} = e^{-(\mathbf{i} r_x + \mathbf{j} r_y + \mathbf{k} r_z) \frac{\theta}{2}} = \cos\left(\frac{\theta}{2}\right) - (\mathbf{i} r_x + \mathbf{j} r_y + \mathbf{k} r_z) \sin\left(\frac{\theta}{2}\right)$ be its inverse,
- ▶ $q_{\mathbf{x}} = \mathbf{i} x_1 + \mathbf{j} x_2 + \mathbf{k} x_3$ be the pure imaginary quaternion of $\mathbf{x} \in \mathbb{R}^3$.

Then

$$q_{\mathbf{x}, \text{rotated}} = q_{\mathbf{r}_0} \cdot q_{\mathbf{x}} \cdot q_{\mathbf{r}_0}^{-1}$$

describes a rotation of $\mathbf{x}$ around $\mathbf{r}_0$ by θ .

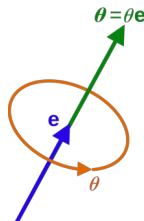
Note: Rotating points this way always involves mapping from/to imaginary quaternions

Instead of quaternions, axis-angle rotations can also be directly expressed:

Theorem (Rodrigues' rotation formula)

Given a unit vector $\mathbf{r}_0 \in \mathbb{R}^3$ ($\|\mathbf{r}_0\| = 1$) and angle $\theta \in \mathbb{R}$, Rodrigues' formula for rotating a vector $\mathbf{x} \in \mathbb{R}^3$ is given by:

$$\mathbf{R}_{\text{axis-angle}} \mathbf{x} = \mathbf{x} + \sin(\theta) (\mathbf{r}_0 \times \mathbf{x}) + (1 - \cos(\theta)) (\mathbf{r}_0 \times (\mathbf{r}_0 \times \mathbf{x}))$$



Proof: Decompose $\mathbf{x}$ into parallel and perpendicular components $\mathbf{x} = \mathbf{x}_{\parallel} + \mathbf{x}_{\perp}$ with:

$$\mathbf{x}_{\parallel} = \langle \mathbf{x} | \mathbf{r}_0 \rangle \mathbf{r}_0$$

$$\begin{aligned} \mathbf{x}_{\perp} &= \mathbf{x} - \mathbf{x}_{\parallel} \\ &= \mathbf{x} - \langle \mathbf{x} | \mathbf{r}_0 \rangle \mathbf{r}_0 \\ &= \underbrace{\langle \mathbf{r}_0 | \mathbf{r}_0 \rangle}_{=1} \mathbf{x} - \langle \mathbf{x} | \mathbf{r}_0 \rangle \mathbf{r}_0 \\ &= -\mathbf{r}_0 \times (\mathbf{r}_0 \times \mathbf{x}) \end{aligned}$$

$$\begin{bmatrix} \mathbf{a} \times (\mathbf{b} \times \mathbf{c}) \\ = \langle \mathbf{a} | \mathbf{c} \rangle \mathbf{b} - \langle \mathbf{a} | \mathbf{b} \rangle \mathbf{c} \end{bmatrix}$$

Applying the rotation on each of these components yields:

$$R_{\text{axis-angle}} \mathbf{x}_{\parallel} = \mathbf{x}_{\parallel}$$

$$R_{\text{axis-angle}} \mathbf{x}_{\perp} = \cos(\theta) \mathbf{x}_{\perp} + \sin(\theta) (\mathbf{r}_0 \times \mathbf{x}_{\perp})$$

With this, we can finally conclude:

$$R_{\text{axis-angle}} \mathbf{x} = R_{\text{axis-angle}} \mathbf{x}_{\parallel} + R_{\text{axis-angle}} \mathbf{x}_{\perp}$$

$$= \mathbf{x}_{\parallel} + \cos(\theta) \mathbf{x}_{\perp} + \sin(\theta) (\mathbf{r}_0 \times \mathbf{x}_{\perp})$$

$$= \mathbf{x} - \mathbf{x}_{\perp} + \cos(\theta) \mathbf{x}_{\perp} + \sin(\theta) (\mathbf{r}_0 \times \mathbf{x}_{\perp})$$

$$= \mathbf{x} - (1 - \cos(\theta)) \mathbf{x}_{\perp} + \sin(\theta) (\mathbf{r}_0 \times \mathbf{x}_{\perp})$$

$$= \mathbf{x} - (1 - \cos(\theta)) \mathbf{x}_{\perp} + \sin(\theta) (\mathbf{r}_0 \times \mathbf{x})$$

$$= \mathbf{x} + (1 - \cos(\theta)) (\mathbf{r}_0 \times (\mathbf{r}_0 \times \mathbf{x})) + \sin(\theta) (\mathbf{r}_0 \times \mathbf{x})$$

$$\left[\mathbf{x} = \mathbf{x}_{\parallel} + \mathbf{x}_{\perp} \right]$$

$$\left[\begin{aligned} \mathbf{r}_0 \times \mathbf{x} &= \mathbf{r}_0 \times (\mathbf{x}_{\parallel} + \mathbf{x}_{\perp}) \\ &= \underbrace{\mathbf{r}_0 \times \mathbf{x}_{\parallel}}_{=0} + \mathbf{r}_0 \times \mathbf{x}_{\perp} \end{aligned} \right]$$

$$\left[\mathbf{x}_{\perp} = -\mathbf{r}_0 \times (\mathbf{r}_0 \times \mathbf{x}) \right]$$

Question: Can we write this in expression as a matrix-vector product to get a rotation matrix R ?

Question: Can we write this in expression as a matrix-vector product to get a rotation matrix R ?

$$\mathbf{R}_{\text{axis-angle}} \mathbf{x} = \mathbf{x} + \sin(\theta) (\mathbf{r}_0 \times \mathbf{x}) + (1 - \cos(\theta)) (\mathbf{r}_0 \times (\mathbf{r}_0 \times \mathbf{x}))$$

→ Yes, by rewriting the cross product

Definition (Cross-Product Matrix)

Given a vector $\mathbf{a} \in \mathbb{R}^3$, the cross-product matrix of $\mathbf{a}$ is defined as:

$$[\mathbf{a}]_{\times} = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix} \in \mathbb{R}^{3 \times 3}$$

In particular, for any vector $\mathbf{b} \in \mathbb{R}^3$: $[\mathbf{a}]_{\times} \mathbf{b} = \mathbf{a} \times \mathbf{b}$.

Thus, Rodrigues' rotation formula can be written in matrix form as:

$$\mathbf{R}_{\text{axis-angle}} = \mathbf{I} + \sin(\theta) [\mathbf{r}_0]_{\times} + (1 - \cos(\theta)) [\mathbf{r}_0]_{\times}^2$$

Observation: Quaternions and Rodrigues' rotation formula both take a unit vector r_0 and an angle θ as input
→ Both representations are in fact "the same"

Proof: Use the following shortcuts to define the unit quaternion q_{r_0} :

$$c = \cos\left(\frac{\theta}{2}\right), \quad s = \sin\left(\frac{\theta}{2}\right)$$

With these shortcuts, the quaternion rotation can also be written in matrix form:

$$\begin{aligned} q_{x, \text{rotated}} &= q_{r_0} \cdot q_x \cdot q_{r_0}^{-1} \\ &= (c + i s r_x + j s r_y + k s r_z) \cdot (i x_1 + j x_2 + k x_3) \cdot (c - i s r_x - j s r_y - k s r_z) \\ &\equiv \underbrace{\begin{pmatrix} 1 - 2s^2(r_y^2 + r_z^2) & -2csr_z + 2s^2r_xr_y & 2csr_y + 2s^2r_xr_z \\ 2csr_z + 2s^2r_xr_y & 1 - 2s^2(r_x^2 + r_z^2) & -2csr_x + 2s^2r_yr_z \\ -2csr_y + 2s^2r_xr_z & 2csr_x + 2s^2r_yr_z & 1 - 2s^2(r_x^2 + r_y^2) \end{pmatrix}}_{R_q} \cdot \underbrace{\begin{pmatrix} i x_1 \\ j x_2 \\ k x_3 \end{pmatrix}}_{x_q} \end{aligned}$$

Thus, we only need to show that R_q is equal to $R_{\text{axis-angle}}$.

Thus, we only need to show that $\mathbf{R}_q$ is equal to $\mathbf{R}_{\text{axis-angle}}$.

$$\mathbf{R}_q = \begin{pmatrix} 1 - 2s^2(r_y^2 + r_z^2) & -2csr_z + 2s^2r_xr_y & 2csr_y + 2s^2r_xr_z \\ 2csr_z + 2s^2r_xr_y & 1 - 2s^2(r_x^2 + r_z^2) & -2csr_x + 2s^2r_yr_z \\ -2csr_y + 2s^2r_xr_z & 2csr_x + 2s^2r_yr_z & 1 - 2s^2(r_x^2 + r_y^2) \end{pmatrix}$$

$$= \mathbf{I} + 2sc \begin{pmatrix} 0 & -r_z & r_y \\ r_z & 0 & -r_x \\ -r_y & r_x & 0 \end{pmatrix} + 2s^2 \begin{pmatrix} -(r_y^2 + r_z^2) & r_xr_y & r_xr_z \\ r_xr_y & -(r_x^2 + r_z^2) & r_yr_z \\ r_xr_z & r_yr_z & -(r_x^2 + r_y^2) \end{pmatrix}$$

Split into
constant (1),
mixed (cs),
squared (s^2)

$$= \mathbf{I} + 2sc [\mathbf{r}_0]_{\times} + 2s^2 [\mathbf{r}_0]_{\times}^2$$

Cross-product
matrix

$$= \mathbf{I} + 2 \sin\left(\frac{\theta}{2}\right) \cos\left(\frac{\theta}{2}\right) [\mathbf{r}_0]_{\times} + 2 \sin^2\left(\frac{\theta}{2}\right) [\mathbf{r}_0]_{\times}^2$$

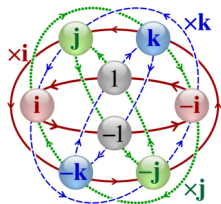
$$= \mathbf{I} + \sin(\theta) [\mathbf{r}_0]_{\times} + (1 - \cos(\theta)) [\mathbf{r}_0]_{\times}^2$$

Double angle
formulas

$$= \mathbf{R}_{\text{axis-angle}}$$

3D Rotation: Quaternions vs. Axis-Angle

Question: Which representation should be used? → Depends on the use case



Quaternions

4 numbers

No singularities

Smooth $\text{slerp}(q_1, q_2, t)$

Fast composition

Applications: Animation, robotics, physics

Axis-Angle

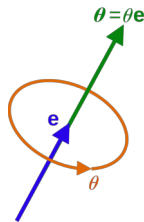
3 numbers
(as rotation vector)

Singularity at $\theta = 0$

Needs different
representation

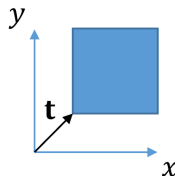
Needs conversion

Applications: SLAM, optimization, CAD



Problem: Unfortunately, translations are **not directly** linear transformations:

$$\underbrace{\begin{pmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{pmatrix}}_{T_t} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ z + t_z \end{pmatrix}$$



→ Unless we switch to a slightly different coordinate representation

Embedding vectors of $\mathbb{R}^n$ into homogeneous coordinates means adding another dimension, typically denoted w . Most common cases are 3D and 2D:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \mapsto \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} \quad \text{or} \quad \begin{pmatrix} x \\ y \end{pmatrix} \mapsto \begin{pmatrix} x \\ y \\ w \end{pmatrix}$$

What does w mean here?

- ▶ $w = 0$: translation or (in mathematical terms) "vector"
- ▶ $w = 1$: point/vertex

Question: What about other homogeneous coordinates, i.e. with $w \notin \{0, 1\}$?

→ For any $c \in \mathbb{R}_{\neq 0}$:

$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \equiv \begin{pmatrix} c \cdot x \\ c \cdot y \\ c \cdot z \\ c \end{pmatrix} \quad \text{as they describe the same point/vertex}$$

A transformation in homogeneous coordinates can be expressed by adding one row and column to the matrix:

$$\mathbf{T}_{\text{homogeneous},3\text{D}} = \left[\begin{array}{ccc|c} & & & t_x \\ & \mathbf{T} & & t_y \\ & & & t_z \\ \hline p_x & p_y & p_z & p_w \end{array} \right]$$

$$\mathbf{T}_{\text{homogeneous},2\text{D}} = \left[\begin{array}{cc|c} & & t_x \\ & \mathbf{T} & t_y \\ \hline p_x & p_y & p_w \end{array} \right]$$

- ▶ t_x, t_y, t_z are the translation components
- ▶ p_x, p_y, p_z, p_w are the projection components

So, a translation by $\mathbf{t} = (t_x, t_y, t_z)^T \in \mathbb{R}^3$ can be written as a *linear transformation in homogeneous coordinates*:

$$\mathbf{T}_{\mathbf{t}} = \left[\begin{array}{ccc|c} & & & t_x \\ & \mathbf{I} & & t_y \\ & & & t_z \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

$$\text{e.g.: } \mathbf{T}_{\mathbf{t}} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{pmatrix}$$

Definition (Rigid transformation)

A transformation T is called *rigid* if it preserves distances, that is

$$\|T(\mathbf{x}) - T(\mathbf{y})\| = \|\mathbf{x} - \mathbf{y}\|$$

for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$.

Types of rigid transformations (including combinations of them):

Rotations

$$\det(\mathbf{R}) = 1$$

Reflections/Mirrorings

$$\det(\mathbf{M}) = -1$$

Translations

$$\left[\begin{array}{ccc|c} & & & \\ & \mathbf{I} & & \mathbf{t} \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

→ Homogeneous coordinates

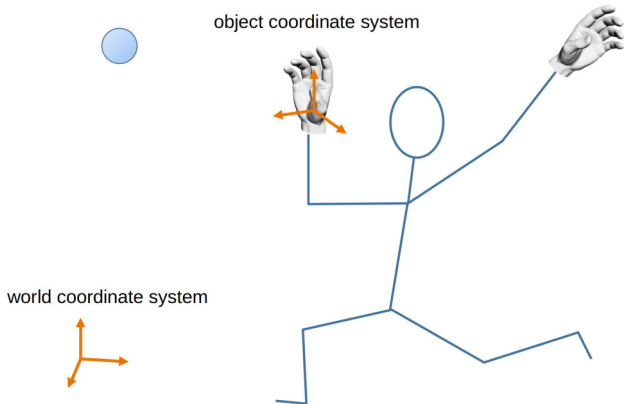
Rigid transformations are crucial in various domains

- ▶ Scene graphs
- ▶ Kinematic chains
- ▶ ...

→ Example of typical task:

Given: Position of ball in *world coordinate system*

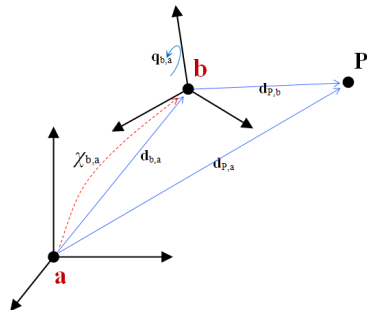
Objective: Coordinates of ball in *object coordinate system*



Given: Point $P_a \in \mathbb{R}^3$ in coordinate system a + Transformation T
(in homogeneous coordinates)

$$T = \left[\begin{array}{c|c|c|c} b_x & b_y & b_z & t \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

- ▶ Columns b_x, b_y, b_z are coordinate axes of b seen from a
- ▶ Translation t is vector pointing from origin of a to origin of b



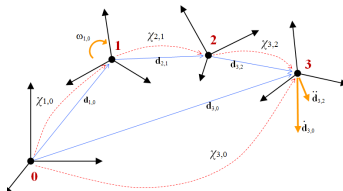
Question: What are the coordinates of the point in b ?

Solution: Transformation affects the coordinates inversely:

$$P_b = T_{a \rightarrow b} \cdot P_a \quad \text{with} \quad T_{a \rightarrow b} = T^{-1}$$

→ Intuitively, if T moves a closer to P , then the coordinates change in the other direction

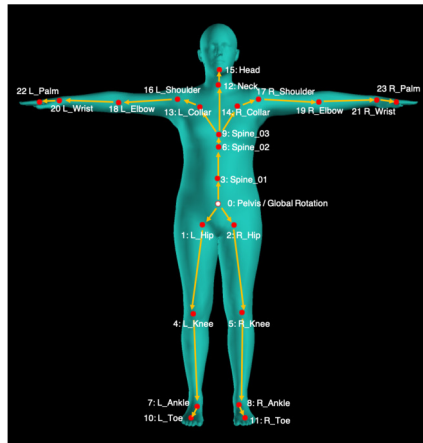
Transformations can be chained to follow a path:



Example: Avatar skeleton (later when discussing human models)

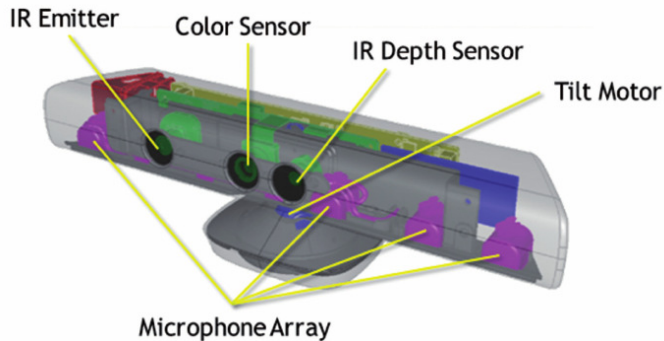
$$T_{\text{world} \rightarrow \text{L_Toe}} = T_{\text{L_Ankle} \rightarrow \text{L_Toe}} \cdot T_{\text{L_Knee} \rightarrow \text{L_Ankle}} \cdot T_{\text{L_Hip} \rightarrow \text{L_Knee}} \cdot T_{\text{Pelvis} \rightarrow \text{L_Hip}} \cdot T_{\text{world} \rightarrow \text{Pelvis}}$$

[avatar position/orientation]



Next Lecture: Cameras

→ **Cameras**



→ **Note:** Initial Exercise *today* with PyTorch introduction & Unity setup!

12:15 - 13:45 in U1.042

